

Database Migration Tool — Project Audit & Roadmap

Date: 19 May 2026 Project Type: Custom Database Migration Tool (Java + Spring Boot + PostgreSQL) Current State: Functional prototype with migration execution, dependency parsing, validation, connection handling, and migration tracking partially implemented.

1. High-Level Project Vision

The project is a custom database migration framework inspired by tools such as:

- Flyway
- Liquibase
- Entity Framework Migrations

The objective of the tool is to:

- Execute SQL migration scripts in version order
 - Track migration history
 - Validate dependencies before execution
 - Handle rollback/down scripts
 - Manage multiple database connections
 - Detect dirty database states
 - Validate checksums
 - Support repeatable migrations
 - Provide CLI + REST APIs
 - Potentially become a reusable enterprise-grade migration framework
-

2. Current Architecture Overview

Backend Stack

- Java
- Spring Boot
- Spring JDBC
- PostgreSQL
- Spring Shell (CLI)
- REST APIs
- JSQLParser (AST SQL parsing)

Important Modules Present

Module	Status	Notes
Migration Engine	Implemented	Core migration execution exists
SQL Executor	Implemented	Executes parsed SQL statements
Migration Loader	Implemented	Loads versioned and repeatable migrations
Dependency Parser	Implemented	Uses AST parsing through JSQLParser
Dependency Validator	Partial	Basic validation exists
Checksum Validation	Partial	Logic exists but incomplete
Dirty State Handling	Partial	Structure exists
Locking System	Partial	Lock service exists
REST APIs	Implemented	Basic migration APIs available
CLI Commands	Partial	Some commands commented out
Connection Management	Implemented	Multi-database handling exists
Logging	Implemented	Log entities and services exist
Rollback Support	Partial	Structure exists but incomplete
Migration Status Tracking	Implemented	Repository + entities exist
Repeatable Migration Support	Partial	Loader supports repeatables
Auto Dependency Recovery	Planned	Not fully implemented

3. Current Folder & Module Breakdown

Core Components

MigrationEngine

Responsibilities:

- Executes migrations
- Coordinates validation
- Runs dependency extraction
- Calls SQL executor
- Updates migration history
- Handles migration lifecycle

Current Status:

✅ Core flow exists ⚠️ Validation partially disabled ⚠️ Error handling still evolving ⚠️ Rollback handling incomplete

MigrationLoader

Responsibilities:

- Reads migration files
- Detects pending migrations
- Handles:
 - Versioned migrations
 - Repeatable migrations
 - Parses filenames
 - Sorts migration order

Patterns detected:

- V001__description.sql
- R__repeatable_name.sql

Current Status:

✅ Well-progressed ✅ Dynamic migration loading exists ✅ Regex parsing exists ⚠️ Needs stronger ordering validation ⚠️ Needs duplicate version protection ⚠️ Needs checksum verification integration

SqlExecutor

Responsibilities:

- Splits SQL statements
- Executes prepared statements
- Runs migration SQL sequentially

Current Status:

✅ Functional ⚠️ No batching optimization ⚠️ Transaction rollback handling needs improvement ⚠️ Better SQL splitting required for edge cases

MigrationValidator

Responsibilities:

- Dirty DB detection
- Checksum validation
- Order validation
- Missing migration detection

Current Status:

⚠ Large portions commented out ⚠ Dirty-state logic appears inverted ⚠ Checksum comparison bug exists
⚠ Sequence validation incomplete

Important Issue Found:

```
if (!repository.existsByDirtyTrue()) {  
    throw new RuntimeException("Database is in DIRTY state. Resolve before  
    continuing.");  
}
```

The condition likely should be:

```
if (repository.existsByDirtyTrue())
```

because the current logic throws when no dirty state exists.

ASTDependencyExtractor

Responsibilities:

- Parses SQL into AST
- Detects:
 - Tables
 - Columns
 - Foreign keys
 - Inserts
 - Updates
 - Select dependencies

Current Status:

✔ Strong advanced feature ✔ JSQLParser integration working ✔ Foreign key dependency extraction exists ⚠ Multi-statement parsing improvement needed ⚠ ALTER TABLE coverage incomplete ⚠ DROP handling incomplete ⚠ Procedure/function parsing absent

This is one of the strongest modules in the project.

DependencyValidator

Responsibilities:

- Validate:
 - Tables
 - Columns
 - Indexes
 - Foreign keys
- Ensure dependencies exist before migration execution

Current Status:

✔ Basic validation works ⚠ Validation responses inconsistent ⚠ Exception handling incomplete ⚠ Auto-recovery not implemented ⚠ Cross-migration dependency resolution pending

4. Existing Features (Completed)

Migration File Handling

Implemented:

- Versioned migration loading
 - Repeatable migration loading
 - SQL file reading
 - Pending migration detection
 - Regex-based filename parsing
-

Database Connection System

Implemented:

- Multiple database connections
- Active connection switching
- Connection persistence
- Dynamic connection selection

Files involved:

- `ConnectionContext`
 - `ConnectionService`
 - `ConnectionRepo`
-

Migration Execution

Implemented:

- Sequential migration execution
 - SQL execution engine
 - Prepared statement execution
 - Migration tracking
 - Execution timing
-

Migration Metadata Tracking

Implemented:

- Migration entity
 - Version tracking
 - Execution history
 - Logs
 - Dirty state structure
 - Checksum structure
-

REST APIs

Implemented:

- Connect database
 - Set active database
 - Migration APIs
 - Status APIs
-

CLI Integration

Implemented:

- Spring Shell integration
- Init command

Partial:

- Status command
 - Migrate command
-

Dependency Analysis

Implemented:

- SQL AST parsing
- Dependency extraction
- Table validation
- Column validation

This is an advanced feature not commonly built in beginner migration systems.

5. Features Currently Partial / In Progress

5.1 Dirty Database Handling

Current:

- Dirty flag exists
- Repository query exists
- Logic incomplete

Needed:

- Mark migration failed when exception occurs
 - Prevent future migrations on dirty DB
 - Add repair command
 - Add manual recovery
-

5.2 Checksum Validation

Current:

- Checksum service exists
- Validation partially implemented

Needed:

- Compare applied checksum vs current file checksum
 - Detect tampered migration files
 - Repair checksum command
 - Validation reporting
-

5.3 Rollback System

Current:

- Down migration idea exists
- Architecture partially prepared

Needed:

- Down migration execution
 - Version rollback
 - Rollback safety checks
 - Transaction rollback integration
 - Partial rollback handling
-

5.4 Migration Locking

Current:

- Lock service exists
- Lock entities exist

Needed:

- Prevent concurrent migration runs
 - Distributed lock handling
 - Lock timeout handling
 - Crash recovery
-

5.5 Transaction Management

Current:

- SQL execution works

Needed:

- Begin transaction
 - Rollback on failure
 - Commit on success
 - Savepoints
 - Per-migration transaction mode
-

5.6 Validation System

Needed:

- Duplicate migration detection
 - Gap detection
 - Out-of-order migration handling
 - Circular dependency handling
 - Cross-script dependency validation
-

6. Missing Major Features

These are the biggest features still left.

Migration Repair Command

Purpose:

Repair broken migration metadata.

Should support:

- Reset dirty state
- Fix checksum mismatch
- Recalculate checksums
- Repair failed migration entries

Priority: HIGH

Rollback / Undo Migrations

Purpose:

Allow reverting schema changes.

Needed:

- Down script support
- Target version rollback
- Automatic rollback execution

Priority: HIGH

Migration Planning Engine

Purpose:

Preview migrations before execution.

Should show:

- What will run
- Dependency chain
- Estimated execution order
- Risk detection

Priority: MEDIUM

Auto Dependency Recovery

You already discussed this feature.

Goal:

If migration order is wrong:

- Scan all migration files
- Find missing CREATE TABLE migration
- Suggest or auto-execute prerequisite migration

This is an advanced intelligent feature.

Priority: HIGH

Repeatable Migration Re-execution Logic

Currently loader supports repeatables.

Still needed:

- Checksum-based rerun detection
- Auto rerun when content changes
- Dependency tracking for repeatables

Priority: MEDIUM

Schema Diff Engine

Future Advanced Feature:

Compare:

- Current DB schema
- Desired schema

Then auto-generate migration scripts.

Priority: FUTURE

Migration Generator

Generate migration files automatically.

Example:

```
migrate create add_users_table
```

Priority: MEDIUM

Migration Dashboard UI

You already have some JavaFX/dashboard files.

Potential features:

- Migration history
- Dirty state view
- Lock view
- Connection management
- Rollback buttons
- Logs viewer

Priority: FUTURE

7. Recommended Development Phases

Phase 1 — Stabilization (Current Priority)

Goal:

Make current system reliable.

Tasks:

- Fix dirty DB logic
- Fix checksum validation bug
- Complete transaction rollback
- Complete migration ordering validation
- Finish pending migration detection
- Improve SQL splitting
- Remove commented dead code
- Add consistent exception handling

Status: IN PROGRESS

Phase 2 — Core Enterprise Features

Goal:

Add production-grade reliability.

Tasks:

- Migration locking
- Rollback support

- Repair command
- Checksum repair
- Retry mechanism
- Better migration metadata

Status: NOT STARTED

Phase 3 — Intelligent Dependency System

Goal:

Build advanced dependency engine.

Tasks:

- Auto dependency resolution
- Migration graph building
- Circular dependency detection
- Auto prerequisite execution
- Dependency visualization

Status: EARLY RESEARCH

Phase 4 — Developer Experience

Goal:

Improve usability.

Tasks:

- Migration generation command
- Better CLI commands
- Better logs
- Colored terminal output
- Migration preview mode
- Dry-run mode

Status: PARTIAL

Phase 5 — Production Readiness

Goal:

Make framework deployable.

Tasks:

- Multi-schema support
- Multi-database support
- Concurrency testing
- Large migration optimization
- Performance benchmarking
- Security hardening
- CI/CD integration

Status: NOT STARTED

Phase 6 — Advanced Features

Goal:

Differentiate from Flyway-like systems.

Tasks:

- Schema diff engine
- Auto migration generation
- Visual dashboard
- AI-assisted dependency recovery
- Migration simulation engine

Status: FUTURE

8. Important Technical Debt

Dead / Commented Code

Several files contain:

- Large commented blocks
- Disabled validations

- Debug print statements

Examples:

- `MigrationValidator`
- `MigrationCommands`
- `MigrationEngine`

Action:

- Refactor
 - Clean architecture
 - Replace `System.out.println`
 - Add structured logging
-

Exception Handling

Current state:

- Many generic `RuntimeException`
- Limited categorized exceptions

Needed:

- Custom exception hierarchy
 - Validation exceptions
 - Dependency exceptions
 - Migration execution exceptions
-

Testing

Currently missing:

- Unit tests
- Integration tests
- Migration simulation tests
- Rollback tests

Priority: VERY HIGH

9. Suggested Immediate Next Tasks

Recommended order:

1. Fix dirty-state logic
 2. Complete checksum validation
 3. Add transaction rollback
 4. Finish rollback migration support
 5. Add repair command
 6. Add migration ordering validation
 7. Add integration tests
 8. Build intelligent dependency recovery
 9. Improve CLI commands
 10. Add migration generator command
-

10. Estimated Current Maturity

Area	Level
Core Execution	Intermediate
Dependency Analysis	Advanced
Reliability	Beginner-Intermediate
Enterprise Readiness	Early Stage
Tool Architecture	Good Foundation
Innovation Potential	High

11. Strongest Parts of the Project

The strongest engineering areas currently are:

AST Dependency Extraction

Using JSQLParser for dependency analysis is a strong architectural decision.

Modular Structure

Project separation is already reasonably good:

- Components
 - Services
 - Validators
 - Parsers
 - DTOs
 - Repositories
-

Multi-Connection Design

The dynamic connection management is a good enterprise-oriented feature.

Migration Metadata Tracking

You already designed:

- Logs
- Dirty state
- Checksums
- Migration history

which are essential for serious migration systems.

12. Final Project Assessment

This is no longer a beginner CRUD project.

You are already building:

- A migration orchestration engine
- A dependency-aware execution system
- A metadata tracking framework
- A partial migration planner

The project is currently between:

- Beginner Flyway clone and
- Early custom enterprise migration framework

The next major leap will happen when:

- Transaction safety becomes stable
- Rollback support is completed
- Dependency auto-resolution is implemented
- Validation becomes reliable
- Testing is added

At that point the project becomes genuinely production-grade.

13. Suggested Future Folder Improvements

Potential cleaner architecture:

```
migration/  
├─ core/  
├─ parser/  
├─ validator/  
├─ executor/  
├─ metadata/  
├─ rollback/  
├─ dependency/  
├─ locking/  
├─ cli/  
├─ api/  
├─ testing/  
└─ recovery/
```

14. Suggested Milestone Tracking

Milestone A

- Stable migration execution
- Stable validation
- Dirty handling
- Transaction rollback

Milestone B

- Rollback support
- Repair command

- Locking
- Repeatable migration stability

Milestone C

- Intelligent dependency recovery
- Migration planning engine
- Graph-based dependency system

Milestone D

- Schema diff engine
 - Migration generation
 - Dashboard UI
 - CI/CD integration
-

15. Conclusion

The project already contains several non-trivial engineering concepts:

- SQL AST parsing
- Dependency analysis
- Migration orchestration
- Multi-database handling
- Metadata tracking
- Checksum architecture
- Dirty-state handling

The foundation is strong.

The biggest priorities now are:

- Reliability
- Validation correctness
- Rollback safety
- Transaction management
- Testing

Once those stabilize, the project can evolve into a genuinely advanced migration framework.